

实验四：基于 RLHF 的大语言模型价值观对齐实验

1. 实验目标

预训练模型虽然具备强大的生成能力，但往往难以遵循人类价值观和指令。本实验旨在通过实践，让学生掌握基于人类反馈的强化学习 (RLHF) 完整流水线。学生需通过训练奖励模型 (RM) 并利用近端策略优化 (PPO) 算法，实现大语言模型从“概率预测”到“人类偏好对齐”的范式转变，确保模型输出满足“有用性、诚实性、无害性”的要求。

2. 实验核心任务

- 偏好数据集处理**：学习处理 (Prompt, Chosen, Rejected) 三元组数据，理解对比学习在建模人类偏好中的应用。
- 奖励模型 (RM) 训练**：实现排序损失函数 (Ranking Loss)，将抽象的人类感官量化为可计算的奖励标量 (Reward Score)。
- PPO 策略对齐**：配置由 Actor、Reference、Reward、Value 组成的四模型架构，在强化学习循环中优化模型行为。
- KL 散度约束分析**：通过实验观察 KL 惩罚项对防止“奖励作弊 (Reward Hacking)”及保持模型语言通用能力的必要性。

3. 实验指南

第一步：偏好数据处理与 RM 初始化

- 任务**：加载并预处理偏好数据集 (如 Anthropic HH-RLHF)。
- 关键点**：构建对比样本对；加载一个较小规模的模型 (如 1B 或者更小的模型) 作为 RM 的基座，在其顶层添加线性分类层用于输出标量分数。

第二步：奖励模型 (RM) 训练

- 任务**：训练一个能够模拟人类打分的“考官”。
- 关键点**：实现 Pairwise Loss 公式，确保模型对 y_w (人类选择) 的评分高于 y_l (人类拒绝)；观察模型在测试集上的分类准确率 (Accuracy)。

第三步：PPO 强化学习循环 (实验重点)

要求学生配置并启动 PPO 训练逻辑，监测以下核心动态：

1. **四模型协同**：理解 Actor (策略模型) 与 Reference (参考模型) 的权值关系，理解 Value (价值模型) 如何通过优势估计辅助训练。
2. **奖励趋势**：记录 env/reward 随训练步数的变化，观察模型是否逐渐学会生成符合偏好的文本。
3. **策略偏移**：记录 objective/kl 指标，观察模型在优化过程中偏离原始分布的程度。

第四步：对齐效果评估与验证

- **逻辑一致性**：验证微调后的模型是否能够有效拒绝不安全指令（如“如何制造危险品”）。
- **鲁棒性测试**：尝试减小 KL 惩罚系数 β ，观察模型是否会出现生成重复乱码但奖励分极高的“奖励作弊”现象。

4. 实验报告提交要求

学生需按以下规范提交实验报告 PDF 及核心代码压缩包：

- **实验配置**：明确 RM 训练的超参数（如 Batch Size, Learning Rate）及 PPO 阶段的 KL 系数、采样温度。
- **结果展示**：展示 RM 训练的 **Accuracy 曲线**。
 - 展示 PPO 阶段的 **Mean Reward 曲线**与 **KL Divergence 曲线**。
- **Case Study**：提供至少 3 组微调前后的对话对比，分析对齐算法如何改变了模型的回答风格 and 安全性。
- **提交方式**：发送至老师邮箱：sdzhao@ir.hit.edu.cn
 - **邮件主题及文件名格式**：[实验四]-学号-姓名.zip

5. 关键代码示例

(1) 奖励模型 (RM) 核心损失函数

Python

```
1 import torch.nn.functional as F
2
3 def compute_ranking_loss(reward_chosen, reward_rejected):
4     # 奖励模型目标：最大化胜者与败者的分差
5     # Loss = -log(sigma(r_w - r_l))
6     loss = -F.logsigmoid(reward_chosen - reward_rejected).mean()
7     return loss
```

(2) PPO 训练器初始化 (基于 TRL 库)

Python

```
1  from trl import PPOConfig, PPOTrainer
2
3  config = PPOConfig(
4      model_name="sft_model_path",
5      learning_rate=1.41e-5,
6      batch_size=32,
7      mini_batch_size=4,
8      gradient_accumulation_steps=4,
9      kl_penalty="kl",
10     target_kl=0.1 # 控制策略偏移的目标 KL 值
11 )
12
13 # 初始化训练器 (需载入训练好的 Reward Model 管道)
14 ppo_trainer = PPOTrainer(
15     config=config,
16     model=actor_model,
17     ref_model=ref_model,
18     tokenizer=tokenizer,
19     dataset=dataset
20 )
```

(3) PPO 训练迭代循环

Python

```
1  for batch in ppo_trainer.dataloader:
2      # 1. 采样 (Rollout) : 生成回答
3      response_tensors = ppo_trainer.generate(batch["input_ids"], **gen_kwa
4
5      # 2. 打分 (Evaluation) : 利用训练好的 RM 给出奖励值
6      texts = [q + r for q, r in zip(batch["query"], batch["response"])]
7      rewards = reward_pipe(texts)
8
9      # 3. 优化 (Optimization) : 更新 Actor 与 Value 模型参数
10     stats = ppo_trainer.step(batch["input_ids"], response_tensors, rewards)
11     ppo_trainer.log_stats(stats, batch, rewards)
```